

1. Introdução

O desafio da Global Solution deste semestre pediu para serem desenvolvidas soluções tecnológicas para uma indústria espacial moderna, um sistema capaz de monitorar uma operação remota, interpretar a telemetria recebida e reagir sozinho quando alguma coisa sai do esperado, fornecendo recomendações técnicas que protejam a vida dos astronautas.

O sistema recebe os dados da missão, monta um diagnóstico do estado geral, dispara alertas em ordem de prioridade, projeta o comportamento futuro da reserva de energia e, no fim, sugere o que a tripulação deveria fazer.

1.1 Objetivo Geral

O objetivo do sistema é desenvolver um sistema de monitoramento básico, num resumo que a tripulação consiga ler em segundos e agir. Lendo quatro fontes de dados da missão, avalia cada subsistema e retorna classificado em seus status do mais grave ao mais leve.

Para além do diagnóstico, o sistema entrega uma fila de alertas ordenada por chegada, aponta o último evento crítico registrado, projeta para o próximo ciclo a reserva de energia usando regressão linear e fecha com uma lista de recomendações priorizadas e também cruza dados de fontes diferentes para flagrar leituras fisicamente impossíveis, que normalmente indicam falha de sensor.

Toda a saída é apresentada no terminal, sem interface gráfica e sem dependências externas.

1.2 Contexto do Cenário

A Aurora Base é uma base experimental instalada em Marte, isolada da rede elétrica e dependente apenas do que consegue gerar e armazenar no próprio local. A energia vem de painéis solares fotovoltaicos e de uma turbina eólica, e tudo o que sobra é guardado num banco de baterias de 500 kWh que

serve de reserva para os períodos sem geração, principalmente a longa noite marciana.

Qualquer queda prolongada na geração ou um consumo mal administrado coloca em risco os módulos que mantêm a tripulação viva.

O recorte escolhido foi o Sol 147, um dia em que vários problemas se acumulam em sequência. A turbina eólica entra offline por manutenção preventiva logo cedo, deixando a base dependente só do solar e em seguida, uma tempestade de poeira reduz a irradiância e derruba a eficiência dos painéis.

Com menos geração, a reserva da bateria começa a cair ao longo do dia, e ainda por cima a qualidade da comunicação com a Terra se degrada num momento em que a base mais precisaria pedir apoio.

É justamente esse empilhamento de eventos que dá ao sistema material para diagnosticar, priorizar e prever.

2. Dados Simulados da Missão

Os dados usados no projeto foram criados através de inteligência artificial descrita a seguir pelo tópico 9, e está de acordo Seção 13 do trabalho da Global Solution, com geração, consumo e variáveis ambientais que se comportam de forma realista ao longo do dia.

A intenção foi montar um cenário com causa e efeito, em que a queda da turbina, a tempestade de poeira e a perda de comunicação se refletem nos números, e não apenas valores soltos.

Toda a telemetria fica organizada em quatro arquivos dentro da pasta data/, três em formato CSV e um em JSON, prontos para serem lidos diretamente pelo sistema.

2.1 Arquivos de Dados Utilizados

O [modulos_status.csv](#) traz consigo as situações operacionais dos oito módulos da base, sendo que para cada módulo há o nome, o status binário (1 ligado, 0 desligado), a criticidade declarada (normal, alerta ou crítico), a prioridade e uma descrição técnica curta. Sendo que é a partir disso que o arquivo reconhece se há algo de errado.

Já o [energia_leituras.csv](#) possui as séries temporais de energia ao longo dos oito horários do Sol, com a geração solar, a geração eólica, o consumo e a reserva da bateria em kWh e em porcentagem. Como a ordem das leituras importa para acompanhar a reserva caindo, esse arquivo é tratado como uma sequência cronológica.

E na [variaveis_ambientais.csv](#) reúne as condições do habitat e do ambiente externo nos mesmos oito horários (temperatura externa e interna, radiação, vento, qualidade da comunicação e pressão interna). Servindo como base para avaliar tanto o ambiente quanto a comunicação, e segue a mesma lógica de série temporal do arquivo de energia.

Por último, o [log_eventos.json](#) registra dez eventos em ordem cronológica, cada um com tipo, módulo afetado, severidade, descrição e ação tomada.

2.2 Status dos Módulos da Base

A base possui oito módulos no total, e cada um deles tem um status, uma criticidade e uma prioridade definida.

O que possui prioridade máxima é o “suporte_vida” (status 1, criticidade crítica, prioridade 1), que controla a pressurização do habitat, sendo basicamente o módulo que mantém a tripulação viva.

Logo após, vem os dois módulos de geração de energia. A “energia_solar” (status 1, criticidade crítica, prioridade 2) são os painéis fotovoltaicos, que durante o dia marciano carregam o peso da geração e a “energia_eolica” (status 0, criticidade alerta, prioridade 3) é a turbina, que está desligada por manutenção

preventiva. Foi tirada de operação depois que o sistema acusou desgaste no rolamento.

Ja a comunicação (status 1, criticidade alerta, prioridade 4) é o canal com a Terra, e começa o dia já com o sinal ruim por causa de interferência atmosférica. O habitat (status 1, criticidade crítica, prioridade 2) cuida da estrutura pressurizada e do controle de temperatura lá dentro.

E por fim, os outros três são menos sensíveis, sendo o laboratório (status 1, criticidade normal, prioridade 5) é onde ficam os experimentos, e teve a prioridade rebaixada de propósito para economizar energia, o armazenamento (status 1, criticidade normal, prioridade 6) guarda mantimentos, peças e o que mais a missão precisa estocar e as baterias (status 1, criticidade crítica, prioridade 2) são o banco de 500 kWh, onde fica guardada toda a reserva de energia da base.

2.3 Inconsistência Proposital nos Dados

Conforme orientações da Global solution, no tópico 7 que tras os dados obrigatórios simulados é necessário: *“Ao menos uma inconsistência proposital nos dados para testar a capacidade de diagnóstico da equipe.”*

A inconsistência que foi plantada no dataset está no horário das 21:00, quando o *“energia_leituras.csv”* registra 18,5 kWh de geração solar. Isso é fisicamente impossível: às 21:00 já é noite marciana, a radiação no *“variaveis_ambientais.csv”* está zerada e, sem irradiância, os painéis não teriam como produzir energia.

O sistema pega essa inconsistência na função *“detectar_anomalias”*, que cruza os dois arquivos e se a radiação naquele horário é zero, ou seja, é noite, mas a geração solar é maior que zero, alguma coisa está errada e a leitura é marcada como anomalia.

A saída do sistema aponta exatamente o horário e o valor suspeito, sugerindo verificar o funcionamento dos sensores. Foi esse cruzamento de fontes diferentes que permitiu flagrar o erro, algo que não apareceria olhando só para o arquivo de energia isolado.

3. Estruturas de Dados Utilizadas

As estruturas de dados que foram utilizadas neste projeto foram escolhidas de acordo com a natureza de cada informação utilizada e o tipo de acesso necessário em cada situação.

3.1 Lista

A lista é uma sequência de estrutura, no qual foi ordenada que armazena elementos indexados por posição.

Neste projeto, ela foi usada para armazenar as séries temporais de geração solar, consumo e temperatura ao longo dos oito horários do sol marciano. A escolha se justifica porque a ordem dos elementos representa a própria sequência cronológica das leituras, permitindo percorrer os dados exatamente na ordem em que foram coletados.

Sendo utilizada no código da seguinte maneira, no modulo de [previsao.py](#):

```
# Prevê a reserva de energia do próximo ciclo a partir dos dados de telemetria
def prever_reserva_energetica(telemetria):
    # Extrai apenas a série da reserva de cada leitura (list comprehension)
    reservas = [item["reserva_bateria_pct"] for item in telemetria]
    # Cria os índices de tempo (0, 1, 2, ...) com o mesmo tamanho da série
    x = list(range(len(reservas)))
    # Calcula a reta de tendência da reserva
    m, b = regressao_linear(x, reservas)
    # Projeta a reserva para o próximo ciclo (o índice seguinte ao último)
    prox_ponto = prever_proximo_valor(m, b, len(reservas))

    # Retorna: histórico completo, previsão do próximo ciclo e tendência (m) m negativo indica reserva caindo; positivo indica subindo
    return reservas, prox_ponto, m
```

3.2 Dicionário

O dicionário é uma estrutura que armazena pares de chave e valor, dando o acesso direto a um dado a partir de sua chave.

Ele foi aplicado para acessar o status e a criticidade de cada módulo do sistema diretamente pelo nome, sem a necessidade de percorrer uma lista inteira, o que torna a consulta mais rápida e direta.

Sendo utilizada no código da seguinte maneira, no módulo de [logica.py](#):

```
#Funcao de avaliacao - Classifica um módulo como normal, alerta, crítico ou desconhecido.
def classificar_modulo(nome, telemetria):
    modulos = telemetria["modulos"]
    # Se o modulo nao existe na telemetria, retorna como desconhecido.
    if nome not in modulos:
        return "desconhecido"

    #Esta puxando os dados de um módulo específico de dentro da telemetria.
    info = modulos[nome]
    #Sendo que 1 significa ligado e 0 desligado
    status = info["status"]
    criticidade = info["criticidade"]
```

3.3 Fila (deque)

A fila segue o princípio FIFO (*First In, First Out*), onde o primeiro elemento inserido é também o primeiro a ser removido.

Neste projeto, foi utilizada para organizar os alertas pendentes por ordem de chegada, garantindo que o alerta mais antigo seja sempre tratado primeiro e respeitando, assim, a prioridade temporal dos eventos.

Sendo utilizada no código da seguinte maneira, no módulo de [logica.py](#):

```

#Fila: FIFO - First In First Out
class Fila:
    def __init__(self):
        # lista vazia que guarda os itens
        self.itens = []

    def enfileirar(self, item):
        # Item novo entra sempre no fim da fila.
        self.itens.append(item)

    def desenfileirar(self):
        # Remove e devolve o item mais antigo, o do início.
        if self.itens:
            return self.itens.pop(0)
        #Se vazia, devolve None.
        return None

```

3.4 Pilha

Já a pilha funciona de forma oposta, seguindo o princípio LIFO (Last In, First Out), em que o último elemento inserido é o primeiro a ser removido.

Ela foi empregada para registrar os eventos críticos analisados, permitindo consultar com facilidade o evento mais recente, o que é especialmente útil para a análise dos últimos acontecimentos do sistema.

Sendo utilizada no código da seguinte maneira, no modulo de [logica.py](#):

```

#Pilha LIFO - Last in First Out
class Pilha:
    def __init__(self):
        # pilha vazia que guarda os itens
        self.itens = []

    def empilhar(self, item):
        # Item novo vai para o topo.
        self.itens.append(item)

```

3.5 Matriz (lista de listas)

A ideia de matriz aparece na forma como as leituras de telemetria ficam organizadas por horário e por variável, em que cada horário funciona como uma linha e cada grandeza medida (radiação, geração solar, consumo, temperatura) como uma coluna. Na prática, essa estrutura bidimensional é representada por

listas paralelas de leituras, a de energia e a de ambiente, percorridas pelo mesmo índice. Assim, o índice indica o horário (a linha) e os campos de cada leitura fazem o papel das colunas, o que permite cruzar grandezas diferentes de um mesmo momento.

Sendo utilizada no código da seguinte maneira, no modulo de [logica.py](#):

```
#Pilha LIFO - Last in First Out
class Pilha:
    def __init__(self):
        # pilha vazia que guarda os itens
        self.itens = []

    def empilhar(self, item):
        # Item novo vai para o topo.
        self.itens.append(item)
```

3.6 Hierarquia da Missão

Toda a telemetria da missão é agrupada num único dicionário aninhado, montado no [main.py](#) antes de seguir para o diagnóstico.

No nível de cima ficam três chaves, módulos, energia e ambiente. Sendo a chave do módulos é um dicionário em que cada nome de módulo aponta para outro dicionário com status, criticidade e prioridade.

As chaves energia e ambiente são listas de leituras, e cada leitura é por sua vez um dicionário com os valores daquele horário. Na prática, o bloco de energia carrega a geração solar, a geração eólica, o consumo e a reserva da bateria, enquanto o bloco de ambiente reúne o que diz respeito ao habitat, temperatura interna e externa, radiação, vento, qualidade da comunicação e pressão.

A combinação de dicionário (acesso por nome) com lista (ordem cronológica) num mesmo objeto foi o que permitiu passar toda a telemetria adiante de uma vez só, sem espalhar variáveis soltas pelo código.

4. Regras Lógicas de Diagnóstico

O diagnóstico da missão é montado a partir de uma série de regras que classificam cada subsistema em um de três estados, sendo eles normal, alerta ou crítico.

Essas regras usam a estrutura como IF, ELIF, ELSE para decidir o rótulo, combinada com os operadores lógicos AND, OR e NOT para expressar as condições.

4.1 Expressão Booleana Principal

Sendo a parte principal do diagnostico, localizado em [logica.py](#), na função `expressao_booleana_principal`, que reduz todo o estado da missão a uma única expressão lógica.

Ela combina os rótulos dos subsistemas mais sensíveis e devolve True quando a base deve entrar em estado crítico:

```
#Se a situação for critica o estado e "critico".
if expressao_booleana_principal(telemetria):
    estado = "critico"
#se houver qualquer alerta pendente na fila, o estado é "alerta"
elif not fila_alertas.vazia():
    estado = "alerta"
#Caso contrario retorna como normal.
else:
    estado = "normal"

#Monta um diagnostico
```

4.2 Regras e Ações do Diagnóstico

A lógica é sempre a mesma em todas as regras, o sistema detecta uma condição nos dados, define se é alerta ou crítico e já sugere o que fazer.

Começando pela energia, ela só vira crítica quando duas coisas acontecem juntas, a reserva abaixo de 25% e o consumo maior que a geração, sendo que a

recomendação é desligar os módulos não essenciais e checar a geração. O alerta é mais simples, dispara quando a reserva cai abaixo de 50% ou quando já tem déficit com a reserva ainda abaixo de 60%, e pede pra ativar o modo economia.

Já na comunicação, se a qualidade do sinal cai abaixo de 30%, já é crítico, e a ação é abrir um canal alternativo. E entra em alerta, quando não é verdade que a qualidade está em 70% ou mais e a recomendação é olhar as antenas e transmissores.

E no ambiente a gente cruza radiação com temperatura, sendo crítico quando a radiação chega a 0,6 mSv e a temperatura está fora dos 18 a 26 °C ao mesmo tempo, pedindo para verificar a radiação e o isolamento do habitat. Se só uma das duas aparece, radiação a partir de 0,3 mSv ou temperatura fora da faixa, é só alerta e a ação é uma inspeção preventiva.

Os módulos também são olhados um por um e quando um módulo crítico aparece desligado, o estado é crítico e o sistema aciona o protocolo daquele módulo.

As duas últimas regras saem da previsão, onde se a reserva projetada para o próximo ciclo fica abaixo de 25%, é crítico e a recomendação é cortar o consumo dos não essenciais e se fica abaixo de 50% mas ainda com tendência de queda, é alerta, e aí a ação é otimizar o uso da reserva.

4.3 Explicação das Regras em Linguagem Simples

Regra 1 - Energia crítica (uso de AND)

Esta regra avalia o subsistema de energia e usa o operador AND para exigir que duas coisas ruins aconteçam ao mesmo tempo, onde o sistema só declara a energia como crítica quando a reserva da bateria está abaixo de 25% E (**AND**) o consumo está maior que a geração.

A lógica é que reserva baixa, sozinha, ainda é administrável se a base estiver gerando mais do que gasta, o que realmente assusta é estar com pouca reserva e ainda por cima perdendo carga e se tiver essas duas condições a

recomendação gerada é desligar os módulos não essenciais e verificar os sistemas de geração.

Regra 2 - Comunicação em alerta (uso de NOT)

A avaliação da comunicação usa o operador NOT de forma explícita para tratar a faixa intermediária do sinal.

Primeiro o sistema verifica se a qualidade está abaixo de 30%, o que já configura estado crítico e se não for tão baixa assim, entra a regra com NOT.

Se NÃO (**NOT**) for verdade que a qualidade está em 70% ou mais, ou seja, se o sinal está na faixa entre 30% e 70%, o estado vira alerta e acima de 70% a comunicação é considerada normal e em alerta, a recomendação é verificar antenas e transmissores; em estado crítico, estabelecer um canal de comunicação alternativo.

Regra 3 - Risco indireto: energia em alerta com comunicação crítica (uso de OR e AND)

A expressão booleana principal eleva a base a estado crítico se a energia, o ambiente ou o suporte à vida estiverem críticos (várias condições ligadas por **OR**), OU se a energia já estiver em alerta e a comunicação estiver crítica ao mesmo tempo.

Esse último trecho com AND modela uma situação de risco indireto, cada variável sozinha não está num valor extremo, mas a combinação de uma base perdendo energia e ficando sem contato com a Terra é justamente o pior momento para um imprevisto, porque a tripulação fica isolada sem conseguir pedir apoio.

5. Alertas Automáticos

O sistema os gera automaticamente os alertas a partir do diagnóstico de cada subsistema e de cada módulo, quando uma avaliação devolve alerta ou crítico, o nome correspondente entra na fila de alertas pendentes, que segue a ordem de chegada (FIFO), de modo que o primeiro problema detectado é o primeiro a aparecer na lista.

Os itens marcados como críticos, além de entrarem na fila, são empilhados numa pilha separada (LIFO), o que permite consultar com facilidade o evento crítico mais recente.

Os eventos críticos vindos do log histórico também são empilhados, com um prefixo que os distingue dos atuais. Assim, a priorização acontece sozinha, a fila preserva a cronologia dos alertas e a pilha guarda o topo do que há de mais grave.

5.1 Níveis de Severidade

O sistema trabalha com três níveis de severidade, sendo eles:

- O nível **CRÍTICO**: acionado quando algum subsistema vital falha de forma grave, ou seja quando a energia crítica (reserva abaixo de 25% com déficit de geração), ambiente crítico (radiação alta junto de temperatura fora de faixa), suporte à vida desligado, ou a combinação de energia em alerta com comunicação crítica. É o nível que exige ação imediata da tripulação.
- O nível **ALERTA**: cobre as situações intermediárias, em que algo já saiu do normal mas ainda há margem para agir, acontece quando a reserva abaixo de 50%, sinal de comunicação entre 30% e 70%, radiação ou temperatura fora da faixa segura, ou um módulo de menor criticidade desligado.
- O nível **NORMAL**: é o estado de rotina, atribuído quando nenhuma das condições de alerta ou crítico se confirma, sendo o estado global da missão recebe o rótulo mais grave aplicável e se a expressão booleana principal aponta situação crítica, o estado é crítico; se não, mas existe qualquer item na fila de alertas, o estado é alerta; caso contrário, normal.

6. Análise e Previsão de Dados

6.1 Variável Analisada e Justificativa

A variável que foi escolhida pra prever foi a reserva da bateria em porcentagem (`reserva_bateria_pct`), acompanhada ao longo dos oito horários do Sol marciano. E não foi uma escolha aleatória, numa base isolada como a Aurora, a reserva da bateria é o que segura os módulos críticos de pé justamente quando não tem geração, tipo durante a longa noite marciana.

Se essa reserva cai abaixo de um limite seguro, o suporte à vida já entra em risco. Por isso ela é a variável que mais vale a pena antecipar. Prever pra onde a reserva está indo permite agir antes do problema virar crítico, em vez de só correr atrás depois que a bateria já esvaziou.

6.2 Técnica Utilizada

A técnica foi regressão linear pelo método dos mínimos quadrados, feita do zero no `previsao.py`, sem Pandas, NumPy ou Scikit-learn. A ideia é achar a reta $y = a \cdot x + b$ que melhor descreve pra onde a reserva está indo ao longo do tempo, onde x é o índice do horário e y é a reserva da bateria naquele momento.

Primeiro o sistema monta a série de reservas e o vetor de tempo. Depois calcula quatro somatórias:

- a soma dos x ;
- a soma dos y ;
- a soma dos produtos $x \cdot y$;
- a soma dos x ao quadrado.

Com essas somatórias, ele acha o coeficiente angular pela fórmula $a = (nSoma_{xy} - Soma_x Soma_y) / (nSoma_{x2} - Soma_x^2)$ e o *intercepto* $b = (Soma_y - aSoma_x) / n$. A previsão do próximo ciclo é só aplicar a reta no índice seguinte ao último.

Como comparação, o mesmo arquivo tem a função `media_movel`, que faz uma estimativa rápida de curto prazo usando a média dos últimos valores da série. Ela serve de contraponto, enquanto a regressão pega a tendência do dia todo, a média móvel responde mais ao que aconteceu por último.

6.3 Resultado e Impacto na Decisão

Com os dados do Sol 147, a série de reservas ficou em [56,0; 52,9; 50,4; 49,5; 50,9; 49,3; 45,8; 45,6], e a regressão devolveu um coeficiente angular de - 1,3119 pontos percentuais por ciclo. Ou seja, a reserva vem caindo em média 1,3 ponto a cada horário. Aplicando a reta ao próximo ciclo, a projeção é de 44,1%.

Ele mexe direto numa recomendação do sistema, como os 44,1% previstos cruzam o limiar de alerta de 50% e a tendência é de queda, o módulo de recomendações dispara a ação "otimizar uso da reserva energética". É exatamente o que o enunciado pedia, a previsão entra na lógica de decisão e gera uma ação preventiva antes da reserva chegar num nível perigoso. Se a queda projetada fosse mais feia, abaixo de 25%, a recomendação subiria pra reduzir o consumo dos módulos não essenciais.

7. Recomendações Geradas pelo Sistema

Tudo vem da função `gerar_recomendacoes`, que cruza o diagnóstico já calculado com a previsão de energia e devolve uma lista ordenada por prioridade. A ordem em que as regras são checadas define a urgência, o suporte à vida vem sempre primeiro, depois os subsistemas (energia, ambiente e comunicação), e por último as recomendações que dependem da projeção da reserva. Se nenhuma condição é satisfeita, o sistema devolve uma mensagem padrão indicando operação normal. Abaixo estão as recomendações organizadas pelos três níveis de prioridade.

Com o dataset do Sol 147, o sistema gerou duas recomendações de prioridade alta.

7.1 Prioridade Crítica

Acionadas quando um subsistema vital está crítico. São as ações de resposta imediata:

- **Suporte à vida crítico:** acionar protocolo de suporte à vida.

- **Energia em estado crítico:** desligar módulos não essenciais e verificar sistemas de geração.
- **Condições ambientais críticas:** verificar radiação e isolamento do habitat.
- **Canais de comunicação comprometidos:** estabelecer canais de comunicação alternativos.
- **Reserva projetada abaixo de 25%:** reduzir consumo dos módulos não essenciais.

No dataset do Sol 147 nenhuma recomendação crítica foi disparada, já que o estado global ficou em alerta e não em crítico.

7.2 Prioridade Alta

Acionadas em estado de alerta, quando ainda há margem para agir de forma preventiva:

- Energia em estado de alerta: ativar modo economia de energia.
- Condições ambientais perigosas: realizar inspeção preventiva do habitat.
- Falhas nos sistemas de comunicação: verificar antenas e transmissores.
- Reserva projetada abaixo de 50% com tendência negativa: otimizar uso da reserva energética.

Foram essas duas as recomendações geradas no Sol 147: ativar o modo economia, por causa da energia em alerta, e otimizar o uso da reserva, por causa da previsão de 44,1% com tendência de queda.

Prioridade Normal

Quando nenhuma condição de alerta ou crítico se confirma, o sistema retorna a recomendação padrão de rotina:

- Sistemas operando normalmente: manter monitoramento de rotina.

8. Como Executar o Sistema

8.1 Pré-requisitos

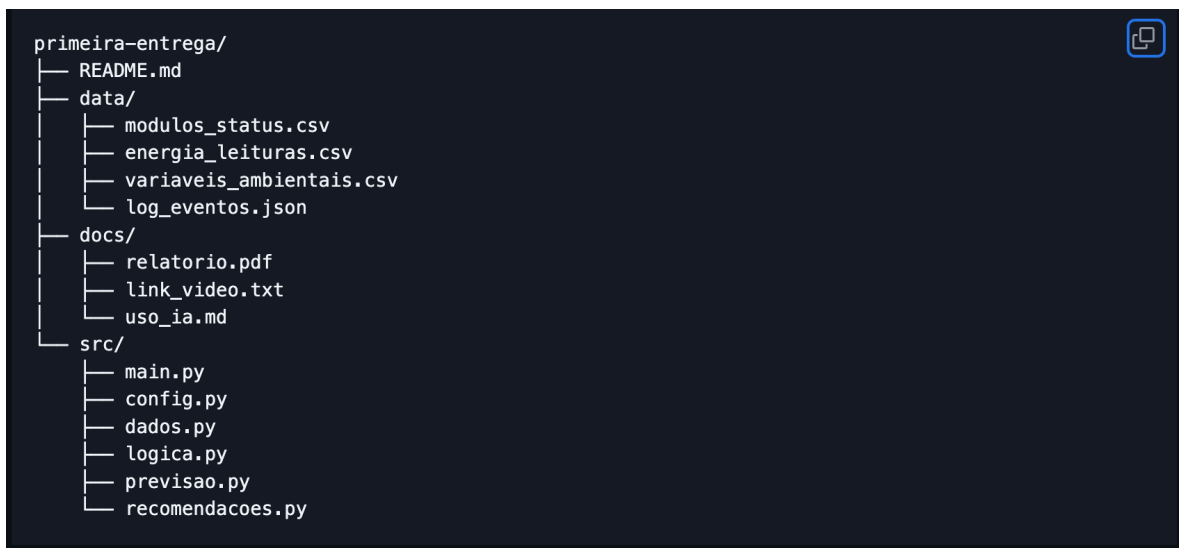
O sistema roda em Python 3.8 ou superior e usa apenas a biblioteca padrão, sem nenhuma dependência externa, nada de Pandas, NumPy, Scikit-learn ou frameworks web. Os únicos módulos usados (csv, json, os e sys) já vêm com o Python. Para funcionar, basta que a pasta de dados contenha os quatro arquivos da telemetria:

- `modulos_status.csv`,
- `energia_leituras.csv`,
- `variaveis_ambientais.csv` e
- `log_eventos.json`.

A saída é gerada inteiramente no terminal, sem interface gráfica.

8.2 Estrutura do Repositório

Abaixo segue a estrutura do repositório:



```
primeira-entrega/  
├── README.md  
├── data/  
│   ├── modulos_status.csv  
│   ├── energia_leituras.csv  
│   ├── variaveis_ambientais.csv  
│   └── log_eventos.json  
├── docs/  
│   ├── relatorio.pdf  
│   ├── link_video.txt  
│   └── uso_ia.md  
└── src/  
    ├── main.py  
    ├── config.py  
    ├── dados.py  
    ├── logica.py  
    ├── previsao.py  
    └── recomendacoes.py
```

8.3 Execução

O projeto roda em Python 3.8 ou superior e usa apenas a biblioteca padrão, sem dependências externas. Da raiz do repositório: `python src/main.py`

Por padrão, o sistema lê os quatro arquivos da pasta data/. Caso queira apontar outra pasta de telemetria, basta usar a flag -d ou --dados: python src/main.py -d /caminho/para/outra/pasta

A pasta indicada precisa conter os arquivos `modulos_status.csv`, `energia_leituras.csv`, `variaveis_ambientais.csv` e `log_eventos.json`.

8.4 Exemplo de Entrada e Saída

A entrada é o pacote de telemetria da pasta data/. O `modulos_status.csv` lista os oito módulos da base, com a turbina eólica offline (status 0) por manutenção preventiva e o canal de comunicação rodando em criticidade alerta.

O `energia_leituras.csv` mostra a reserva caindo de 56% no começo do sol marciano para 45,6% no final, e inclui a leitura suspeita de 18,5 kWh de geração solar às 21h, plantada de propósito. O `variaveis_ambientais.csv` traz a qualidade de comunicação entre 52% e 71%, radiação chegando a 0,71 mSv ao meio-dia e temperatura interna estável em torno de 21 a 22 graus. O `log_eventos.json` reúne dez eventos em sequência narrativa do Sol 147, começando pela queda da turbina, passando pela tempestade de poeira que reduz a geração solar e pela degradação da comunicação, até o diagnóstico noturno.

A saída produzida pelo sistema com essa entrada é:

```
=====
                        DIAGNÓSTICO DA MISSÃO
=====

Estado global: ALERTA

Energia ..... alerta
Comunicação . normal
Ambiente .... normal

Módulos:
- suporte_vida      normal
- energia_solar     normal
- energia_eolica    alerta
- comunicacao       alerta
- habitat           normal
- laboratorio        normal
- armazenamento     normal
- baterias          normal

Alertas pendentes (FIFO):
-> energia
-> modulo:energia_eolica
-> modulo:comunicacao

Último evento crítico: historico:comunicacao
```

```
=====
                        PREVISÃO DE RESERVA ENERGÉTICA
=====

Histórico (% de bateria):
ciclo 0: 56.0%
ciclo 1: 52.9%
ciclo 2: 50.4%
ciclo 3: 49.5%
ciclo 4: 50.9%
ciclo 5: 49.3%
ciclo 6: 45.8%
ciclo 7: 45.6%

Próximo ciclo previsto: 44.1%
Tendência: -1.3119 pp/ciclo (caindo)
```

```
=====
                        RECOMENDAÇÕES
=====

1. Energia em estado de alerta, ativar modo economia de energia
2. Reserva energética em alerta, otimizar uso da reserva energética

=====
                        ANOMALIAS DETECTADAS
=====

1. Anomalia detectada às 21:00: geração solar de 18.5 kWh durante a noite (radiação zero)

>> Verificar funcionamento dos sensores
```

9. Uso de Inteligência Artificial

Para o uso da Inteligência Artificial (IA), foram seguidas as diretrizes estabelecidas na Seção 13 do trabalho da Global Solution. A IA foi utilizada com cautela pela equipe, sendo permitida apenas para organizar ideias, revisar textos, explicar conceitos e gerar dados simulados. Fica vedada sua utilização para a solução de código, análises e conclusões.

Todo e qualquer uso da IA foi rigorosamente revisado pela equipe antes de ser incorporado ao projeto.

9.1 Onde a IA foi utilizada

Para começar este projeto, foi necessária a preparação dos dados de telemetria, resultando na geração de dados em quatro arquivos, três no formato CSV e um no formato JSON, seguindo os requisitos estabelecidos na Seção 7 do enunciado. Foi inserida, de forma proposital, uma inconsistência nos dados para testar o sistema de diagnóstico.

O prompt utilizado foi:

“Atue como engenheiro aeroespacial responsável pela telemetria da missão Aurora Base, Sol 147, uma base experimental em Marte alimentada por energia solar fotovoltaica e turbina eólica em sistema off-grid com banco de baterias. Gere os 4 arquivos de dados simulados de telemetria descritos abaixo. Os dados serão usados por um sistema estudantil de monitoramento em Python puro, então precisam estar prontos pra salvar e usar diretamente.

ARQUIVO 1 — `modulos_status.csv`

Monte o CSV com o status operacional dos 8 módulos da base. Use os valores exatos abaixo:

suporte_vida: status 1, criticidade critico, prioridade 1

energia_solar: status 1, criticidade critico, prioridade 2

energia_eolica: status 0, criticidade alerta, prioridade 3 (offline por manutenção)

comunicacao: status 1, criticidade alerta, prioridade 4 (sinal degradado)

habitat: status 1, criticidade critico, prioridade 2

laboratorio: status 1, criticidade normal, prioridade 5

armazenamento: status 1, criticidade normal, prioridade 6

baterias: status 1, criticidade critico, prioridade 2

Colunas: modulo, status, criticidade, prioridade, descricao Adicione uma descrição técnica curta pra cada módulo.

ARQUIVO 2 — energia_leituras.csv

Gere leituras de energia nos horários aleatórios com intervalos de 3 horas, representando um sol marciano.

Colunas: horario, geracao_solar_kwh, geracao_eolica_kwh, consumo_kwh, reserva_bateria_kwh, reserva_bateria_pct

Regras de geração e consumo:

geracao_eolica_kwh = 0.0 em todos os horários (turbina offline)

geracao_solar_kwh segue curva de irradiância marciana: zero à noite (00:00 e 03:00), crescendo da manhã até o pico ao meio-dia e caindo à tarde consumo segue o ciclo de atividade da base: maior durante o expediente (09:00–15:00), menor à noite.

Cálculo da reserva:

Capacidade total do banco de baterias: 500 kWh

Reserva inicial: 280 kWh (56%)

$reserva_bateria_kwh(t) = reserva(t-1) + geracao_solar + geracao_eolica - consumo$

*$reserva_bateria_pct = (reserva_kwh / 500) * 100$, com 1 casa decimal*

Calcule e mostre os valores de todos os 8 horários

Preciso que insira UMA inconsistência plausível que a equipe precisará detectar. Não comente qual é a inconsistência, ela deve ser descoberta pelo sistema.

ARQUIVO 3 — variaveis_ambientais.csv

Gere leituras ambientais nos mesmos 8 horários do Arquivo 2.

Colunas: horario, temp_externa_c, temp_interna_c, radiacao_msv, vento_ms, qualidade_comunicacao_pct, pressao_interna_pa

Parâmetros ambientais marcianos:

temp_externa_c: ciclo diurno realista, mínima entre -70°C e -80°C na madrugada, máxima entre -20°C e -25°C ao meio-dia

temp_interna_c: estável entre 19°C e 23°C (habitat pressurizado com controle térmico ativo)

radiacao_msv: 0.0 à noite, cresce a partir das 06:00, pico de 0.65–0.75 mSv ao meio-dia, cai à tarde

vento_ms: entre 8 e 28 m/s com variação ao longo do dia (ventos marcianos irregulares)

qualidade_comunicacao_pct: entre 52% e 72% (módulo em criticidade alerta, sinal degradado por interferência atmosférica)

pressao_interna_pa: 101325 Pa fixo (base pressurizada em 1 atm)

Preencha os 8 horários com progressão coerente com o ciclo dia/noite marciano.

ARQUIVO 4 — log_eventos.json

Gere um JSON de log de eventos do Sol 147 com exatamente 10 registros em ordem cronológica. Os eventos devem contar uma história com causa e efeito ao longo do dia — cada evento deve ser consequência ou contexto do anterior.

Estrutura do JSON:

```
{ "missao": "Aurora Base - Sol 147", "data_sol": "2031-03-15",  
  "total_eventos": 10, "eventos": [ lista aqui ]}
```

Cada evento:

```
{ "id": 1 a 10, "timestamp": "2031-03-15 HH:MM:SS", "tipo": tipo do evento,  
  "modulo_afetado": nome do módulo, "severidade": "normal", "alerta" ou  
  "critico", "descricao": descrição técnica do que o sensor detectou,  
  "acao_tomada": resposta automática do sistema supervisorio, "resolvido":  
  true ou false }
```

Tipos válidos: ALERTA, FALHA_SENSOR, REINICIALIZACAO, MUDANCA_PRIORIDADE, MODO_ECONOMIA, INFO

Narrativa do dia (causa e efeito):

MODO_ECONOMIA - baterias - normal - baterias atingem nível crítico após noite sem geração solar - resolvido: true

REINICIALIZACAO - comunicacao - alerta - perda de sinal com a Terra, transmissor reiniciado - resolvido: true

FALHA_SENSOR - energia_eolica - alerta - sensor de vibração detecta desgaste no rolamento, turbina desligada para manutenção preditiva - resolvido: false

ALERTA - energia_solar - alerta - tempestade de poeira reduz irradiância nos painéis em ~15% - resolvido: false

MUDANCA_PRIORIDADE - laboratorio - normal - prioridade do laboratório reduzida para proteger módulos críticos com turbina offline - resolvido: true

ALERTA - comunicacao - critico - qualidade do sinal cai abaixo de 50%, protocolo de comunicação de emergência ativado - resolvido: false

MODO_ECONOMIA - baterias - normal - recarga solar eleva reserva acima do limiar seguro, modo de economia desativado - resolvido: true

FALHA_SENSOR - suporte_vida - alerta - sensor de CO₂ registra leitura fora da faixa operacional esperada, marcado como suspeito - resolvido: false

FALHA_SENSOR - energia_solar - alerta - sistema detecta leitura de geração solar durante período noturno, valor inconsistente com irradiância esperada - resolvido: false

INFO - SISTEMA - normal - ciclo de diagnóstico automático concluído com resumo operacional do Sol 147- resolvido: true

Para cada evento, escreva:

descricao: log técnico do sistema, com valores numéricos específicos onde fizer sentido

acao_tomada: resposta automática do sistema supervisorio, como um SCADA real agiria

resolvido: siga os valores indicados acima

Entregue os 4 arquivos separados, cada um com seu nome como título, prontos para salvar.”

A saída foi:

“Os 4 arquivos da missão Aurora Base — Sol 147 estão prontos. Aqui está um resumo das decisões técnicas tomadas em cada um:

modulos_status.csv — 8 módulos com status, criticidades e descrições conforme especificado. energia_eolica offline (status 0), comunicacao em alerta.

energia_leituras.csv — Curva solar marciana realista com pico de 38.4 kWh ao meio-dia, geração eólica zerada em todos os horários, e o balanço acumulado da bateria calculado passo a passo a partir dos 280 kWh iniciais. A inconsistência às 21:00 está embutida silenciosamente (18.5 kWh de geração solar durante a noite).

variaveis_ambientais.csv — Ciclo térmico externo variando de -79°C de madrugada a -22.8°C ao meio-dia. Radiação UV seguindo a irradiância solar. Qualidade de comunicação entre 52–71%, temperatura interna estável em ~21-22°C, pressão fixa em 101325 Pa.

log_eventos.json — 10 eventos em cadeia de causa e efeito ao longo do dia: turbina cai → painel solar perde eficiência pela tempestade → comunicação degrada → sistema entra em modo de economia → sensores suspeitos acumulam → diagnóstico noturno fecha o Sol 147.

Descrições com valores numéricos específicos e respostas SCADA realistas.”

Os arquivos elaborados estão disponíveis no GitHub, cujo link se encontra ao final do trabalho.

A IA também foi usada para ajudar a organizar a estrutura do relatório e do README, partindo dos tópicos que o enunciado pedia. O prompt foi simples:

“Claude, bom dia, tenho que escrever o relatório de um trabalho de faculdade sobre um sistema em python que monitora uma base em marte.

ele lê uns arquivos de dados, faz um diagnóstico, gera alertas, prevê a reserva de energia e dá recomendações. me ajuda a montar uma ordem de tópicos pro relatório e pro readme? Quero que fique fácil de seguir. Poderia me entregar em um DOC que seja fácil de usar”

E a resposta foi:

“Bom dia. Vou montar a estrutura do relatório e do README do Aurora Base num documento Word fácil de seguir. Vou criar o documento Word com a estrutura do relatório e do README. Ran 3 commands, created a file, read a file. Pronto! O documento tem duas partes: a estrutura do relatório (11 tópicos, da capa às referências) e a do README (9 tópicos, mais enxuto e voltado a quem vai rodar o código). Cada item traz uma frase indicando o que escrever, então é só substituir pelo conteúdo do Aurora Base.

Tem índice automático e os tópicos estão como cabeçalhos, então o Word atualiza o sumário sozinho (clica nele → "Atualizar campo"). O coração do relatório é o tópico 6 (Metodologia), onde entra a justificativa da regressão linear + média móvel.

Quer que eu já preencha algum tópico com o conteúdo real do seu projeto, ou prefere ir escrevendo você mesma?”

A ordem dos tópicos foi aproveitada como ponto de partida, mas a sugestão do dashboard web foi descartada, já que o enunciado pede saída apenas no terminal, sem interface gráfica nem frameworks web.

E em outro momento a IA foi usada para montar um checklist com tudo o que o trabalho precisava ter, de modo a não deixar nenhum requisito de fora. O prompt foi:

“Com base no enunciado da Global Solution que vou colar abaixo, monte um checklist completo e organizado por categorias (entrega, repositório, código, dados, estruturas de dados, regras lógicas, alertas, previsão e vídeo) com todos os itens obrigatórios que o trabalho precisa cumprir.

Quero usar isso pra dividir as tarefas da equipe e conferir item por item antes de entregar.”

A resposta foi o checklist abaixo:

“Entrega

- *Arquivo .TXT submetido na plataforma da faculdade com os dois links*
- *Link do repositório GitHub público — acessível sem login*
- *Link do vídeo no YouTube como 'Não Listado' — acessível sem login*

Repositório GitHub

- *Repositório está público (não privado)*
- *Pasta src/ com o arquivo sistema.py*
- *Pasta data/ com os 4 arquivos de dados (csv e json)*
- *Pasta docs/ com uso_ia.md preenchido (se usou IA)*
- *README.md com todos os campos obrigatórios: nome da equipe e RMs, resumo do problema e cenário, estruturas de dados usadas e por quê, regras lógicas e expressão booleana, técnica de previsão e resultado, como executar (python src/sistema.py), exemplo de entrada e saída, recomendações geradas, link do vídeo, conclusões e aprendizados*

Código Python

- *Executa sem erros com: python src/sistema.py*
- *Sem interface gráfica — saída apenas no terminal*
- *Sem Pandas, NumPy, Scikit-learn, Streamlit ou frameworks web*
- *Dados lidos de arquivo externo ou embutidos e documentados*
- *Variáveis 0/1 usadas para status dos módulos*
- *Tabela de status exibida: normal, alerta ou crítico*
- *Pelo menos uma regra baseada em faixas de segurança ou estados binários*

Dados Simulados

- *Status binário de pelo menos 6 módulos críticos*
- *Leituras de energia em pelo menos 6 horários com reservas energéticas*
- *Variáveis ambientais: temperatura, radiação, vento ou comunicação*
- *Log com pelo menos 8 eventos*
- *Tipos de evento: alertas, reinicializações, falhas de sensor, mudanças de prioridade ou modos de economia*
- *Ao menos uma inconsistência proposital detectável pelo sistema*

Estruturas de Dados

- *Lista para séries temporais*
- *Fila (deque) para alertas por ordem de chegada — FIFO*
- *Pilha para eventos críticos analisados — LIFO*
- *Dicionário para acesso de módulos pelo nome*

- *Matriz (lista de listas) para leituras por horário e variável*
- *Hierarquia aninhada representando energia e habitat*

Regras Lógicas

- *IF, ELIF e ELSE para classificar o estado da missão*
- *Operador AND em pelo menos uma regra*
- *Operador OR em pelo menos uma regra*
- *Operador NOT em pelo menos uma regra*
- *Cada regra explicada em linguagem simples*

Alertas e Recomendações

- *Alertas gerados para módulo offline, energia baixa e comunicação comprometida*
- *Alertas classificados em normal, alerta e crítico*
- *Alertas exibidos com os mais críticos primeiro*
- *Recomendação automática para cada alerta*

Análise e Previsão

- *Técnica manual sem Pandas, NumPy ou Scikit-learn*
- *Regressão linear, média móvel ou extrapolação de tendência*
- *Dados, metodologia e resultado documentados*
- *Previsão influencia pelo menos uma recomendação do sistema*

Vídeo de Apresentação

- *Publicado no YouTube como 'Não Listado'*
- *Link funciona sem login*
- *Mostra o sistema em execução no terminal*
- *Apresenta estruturas de dados e regras lógicas*
- *Apresenta a técnica de previsão e o resultado*
- *Todos os integrantes aparecem ou são mencionados*

Esse checklist acabou virando o norte da equipe. Foi a partir dele que dividimos quem ia ficar com cada parte e nos organizamos para fechar tudo dentro do prazo, riscando os itens conforme iam sendo concluídos.

Já no fim do trabalho, antes de fechar a entrega, a IA foi usada mais uma vez para verificar se estava tudo completo. O prompt foi:

“Vou te passar o checklist do trabalho e o que a gente já fez. Confere se está tudo completo ou se ainda falta alguma coisa antes de entregar.”

A resposta foi:

“Pelo que você descreveu, o trabalho cobre todos os itens obrigatórios do checklist: as cinco estruturas de dados estão usadas e justificadas, as regras com AND, OR e NOT estão presentes, a inconsistência é detectada pelo sistema, a previsão influencia uma recomendação e os quatro arquivos de dados estão prontos. Está completo para entrega. Como sugestão extra, vocês poderiam adicionar testes automatizados com pytest para cada módulo e até integrar um pipeline de CI no GitHub que roda esses testes a cada commit, o que deixaria o projeto num nível bem profissional.”

A confirmação de que o trabalho estava completo nos deu segurança para a entrega. A sugestão dos testes automatizados e do pipeline de integração contínua, embora interessante, ficou de fora por fugir do escopo pedido e do tempo disponível para esta entrega.

9.2 Onde a IA NÃO foi utilizada

Para que não fique dúvidas, o trabalho é feito pela equipe e a IA não participou das partes centrais do projeto.

Sendo que o código Python (src/sistema.py) foi escrito integralmente pela equipe, com base nos conceitos das Fases 1, 2 e 3 e a lógica das regras de diagnóstico foi modelada pela equipe a partir da interpretação dos dados, e a escolha e a justificativa das estruturas de dados foram decisões de projeto nossas.

A análise dos resultados e as conclusões do relatório são reflexões críticas da equipe, e a identificação da inconsistência no dataset foi descoberta por inspeção da própria equipe.

9.3 Validação crítica realizada

Esta parte do registro foi onde precisávamos olhar o que a IA entregou sem aceitar nada de cara. Nada gerado por IA entra no projeto sem passar pela conferência de pelo menos duas pessoas e na prática, isso significou abrir os arquivos, calcular alguns valores à mão e cruzar uma fonte com a outra antes de dar qualquer coisa por boa.

A inconsistência plantada nos dados de energia está no horário das 21:00: o arquivo `energia_leituras.csv` registra 18,5 kWh de geração solar em pleno período noturno. Isso contradiz frontalmente o `variaveis_ambientais.csv`, que marca radiação 0,0 mSv naquele horário, e ainda casa com o nono evento do `log_eventos.json` (FALHA_SENSOR em energia_solar, “leitura de geração solar durante período noturno”). Não havia Sol em Marte às nove da noite, então número não podia existir.

A IA não nos contou onde estava o erro, justamente como pedimos e ela só avisou que havia uma inconsistência embutida. Foi o nosso sistema, cruzando as três fontes (geração de energia, radiação ambiental e log de eventos), que apontou a contradição.

Se tivéssemos confiado apenas no resumo que a ferramenta escreveu, teríamos aceitado a explicação pronta sem realmente entender por que o valor estava errado.

Para o README, foi conferido item por item, se todas as seções exigidas pelo enunciado (Seção 11) estavam contempladas na estrutura sugerida e completamos as que a ferramenta não previu. Cada espaço-reservado (placeholder) foi substituído por conteúdo real do projeto. Nenhuma frase-guia genérica ficou no texto final. A tabela de estruturas de dados foi reescrita do zero, com as escolhas reais do projeto e a justificativa de cada uma. A versão inicial sugerida pela IA trazia estruturas que não faziam sentido para o nosso problema, e essas foram simplesmente descartadas.

Já o checklist gerado pela IA foi útil para organizar a divisão de tarefas, mas a equipe não o tratou como verdade pronta. Conferimos cada item contra o enunciado original, porque um checklist é só uma releitura do que a ferramenta entendeu do texto e o que ela entende nem sempre é o que o enunciado de fato exige. Nessa conferência percebemos que alguns itens precisavam de ajuste de redação para refletir com exatidão os requisitos das Seções 7 e 11, e foi a versão revisada por nós, e não a original da IA, que virou o guia da equipe.

O mesmo cuidado valeu para o double-check feito antes da entrega. A IA respondeu que estava "tudo completo", mas é importante registrar o que essa confirmação significava de verdade, sendo que ela avaliou apenas o que nós descrevemos a ela em texto, sem nunca ter rodado o código nem aberto os arquivos do repositório. Ou seja, a ferramenta confirmou a nossa descrição, não o projeto.

Por isso, a checagem que realmente contou foi a que a equipe fez manualmente, executando `python src/sistema.py`, abrindo os quatro arquivos de dados e cruzando cada item do checklist com o que estava de fato implementado.

A confirmação da IA serviu como um reforço de tranquilidade, mas a responsabilidade pela conferência continuou sendo nossa.

9.4 Limitações observadas no uso de IA

Ao longo do trabalho, ficou claro para a equipe que a IA acerta na forma com muito mais facilidade do que no conteúdo. Ela organiza, formata e escreve com fluência, mas isso não é o mesmo que estar certa. Algumas limitações que sentimos na pele.

Confiança que não corresponde à exatidão. No double-check final, a ferramenta confirmou com toda a tranquilidade que estava "tudo completo", mas baseada apenas no que nós descrevemos a ela, e não no código rodando de fato. Foi a equipe que abriu o terminal, executou o sistema e conferiu cada item contra o enunciado. A IA não roda o projeto, ela simplesmente acredita no que você conta.

Sugestões fora do escopo. Logo depois de confirmar a entrega, ela sugeriu adicionar testes automatizados com `pytest` e um pipeline de integração contínua no GitHub. É uma boa prática de engenharia, mas o enunciado não pedia nada disso, e seguir a sugestão às cegas teria inflado o trabalho com coisas que não eram avaliadas, consumindo tempo que não tínhamos. Foi preciso conhecer o escopo para saber dizer "não" a uma sugestão tecnicamente correta.

Tendência a propor o caminho “mais técnico”. Tanto na estrutura do relatório (onde sugeri um dashboard web) quanto na conferência final, a ferramenta puxava para soluções mais robustas do que o problema exigia. O enunciado pedia saída apenas no terminal, em Python puro e cabia a nós manter o projeto nesse limite.

Dados que parecem certos, mas precisam ser recalculados. Os arquivos de telemetria vieram bem formatados e plausíveis, mas “plausível” não é “verificado”. Refizemos o balanço acumulado da bateria à mão para confirmar que os números fechavam a partir dos 280 kWh iniciais.

9.5 Reflexão final da equipe sobre o uso de IA

A lição que levamos deste trabalho é que a IA é uma ferramenta para facilitar, e não para resolver. Ela auxilia e tira do nosso caminho o que é repetitivo, gerar um dataset coerente, montar um esqueleto de relatório, listar requisitos a partir de um enunciado e isso é valioso e economiza tempo e assim focar no que realmente importa.

Mas em nenhum momento ela substituiu esse pensamento.

Aprendemos também que mesmo as tarefas seguras, como fazer um checklist e depois um double-check, precisam ser conferidas.

A IA montou um checklist ótimo e confirmou nossa entrega, mas quem garantiu que cada item estava de fato cumprido fomos nós, comparando linha por linha com o enunciado e rodando o código.

Confiar que estava tudo certo dela sem verificar seria abrir mão justamente da parte que prova que entendemos o trabalho, sendo uma ferramenta que auxilia, mas não faz.

Outra coisa que ficou evidente é que não basta copiar e colar, é preciso conhecer. Só conseguimos descartar a sugestão do dashboard, recusar o pytest e identificar a inconsistência das 21:00 porque definimos o escopo e os dados.

Quem não conhece o problema aceita qualquer resposta bem escrita, e respostas bem escritas e erradas são exatamente o tipo de armadilha que a IA produz com naturalidade.

Percebemos ainda, na prática, que prompt vago gera resposta genérica, quanto mais específicos fomos nos requisitos, mais útil foi o retorno.

10. Conclusões e Aprendizados

Ao final do projeto, o sistema conseguiu monitorar e diagnosticar todos os subsistemas da Aurora Base, classificando o estado global, organizando os alertas por prioridade, projetando a reserva de energia e detectando a inconsistência que tinha nos dados. O ciclo inteiro, da leitura dos arquivos até a geração das recomendações, rodou sem erros e entregou uma saída clara no terminal.

A decisão que mais facilitou nossa vida foi separar o código em módulos. Cada parte ficou possível testar sozinhos e com isso virou só uma questão de editar uma constante no config.py, em vez de caçar a regra escondida no meio de outra função. A escolha das estruturas de dados também passou a fazer muito mais sentido quando cada uma resolveu um problema concreto.

O dicionário deu acesso direto ao status de um módulo pelo nome, a lista preservou a ordem cronológica das leituras, a fila organizou os alertas pendentes na ordem de chegada e a pilha deixou fácil consultar o último evento crítico.

A previsão por regressão linear, ajudou bastante para antecipar a queda da reserva e disparar uma recomendação preventiva. Com os 44,1% projetados para o próximo ciclo cruzando o limiar de alerta, o sistema sugeriu otimizar o uso da reserva antes da situação piorar, o que mostra que a previsão não ficou só no papel: ela mexeu de fato numa decisão.

Enquanto isso a parte que deu mais trabalho foi fazer o sistema desconfiar dos dados em vez de simplesmente processá-los. A inconsistência da geração solar à noite só apareceu porque o código cruzou dois arquivos diferentes, o de energia e o de ambiente. Foi um lembrete prático de que sensor falha e de que, numa operação real, ignorar uma leitura suspeita poderia gerar uma previsão otimista demais e levar a decisões erradas sobre consumo.

No fim, o projeto ligou diretamente os conceitos das aulas (estruturas de dados, lógica booleana e regressão linear) a uma aplicação concreta, em que cada escolha tinha sua justificativa.

Ficou claro que projetar um sistema desses é, antes de tudo, um exercício de prioridades, sendo o suporte à vida vem primeiro, a transparência dos alertas vem logo atrás e a decisão final é sempre da tripulação.

O software ajuda a olhar para muitos dados ao mesmo tempo, mas não substitui o julgamento humano.

11. Links de Entrega

- GITHUB - <https://github.com/GS-FIAP-CC/primeira-entrega>
- Youtube